



FIREBRAND

LYCEUM EDUCATION GROUP

Data Structures and Their Application

C# Pathway | Session 4

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values

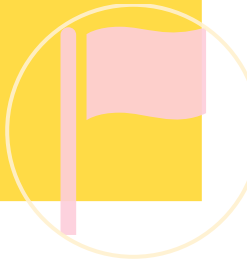


Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Data Structures and Their Application

Lesson Objectives

Apply

- Apply appropriate data structures to store and organise data

Analyse

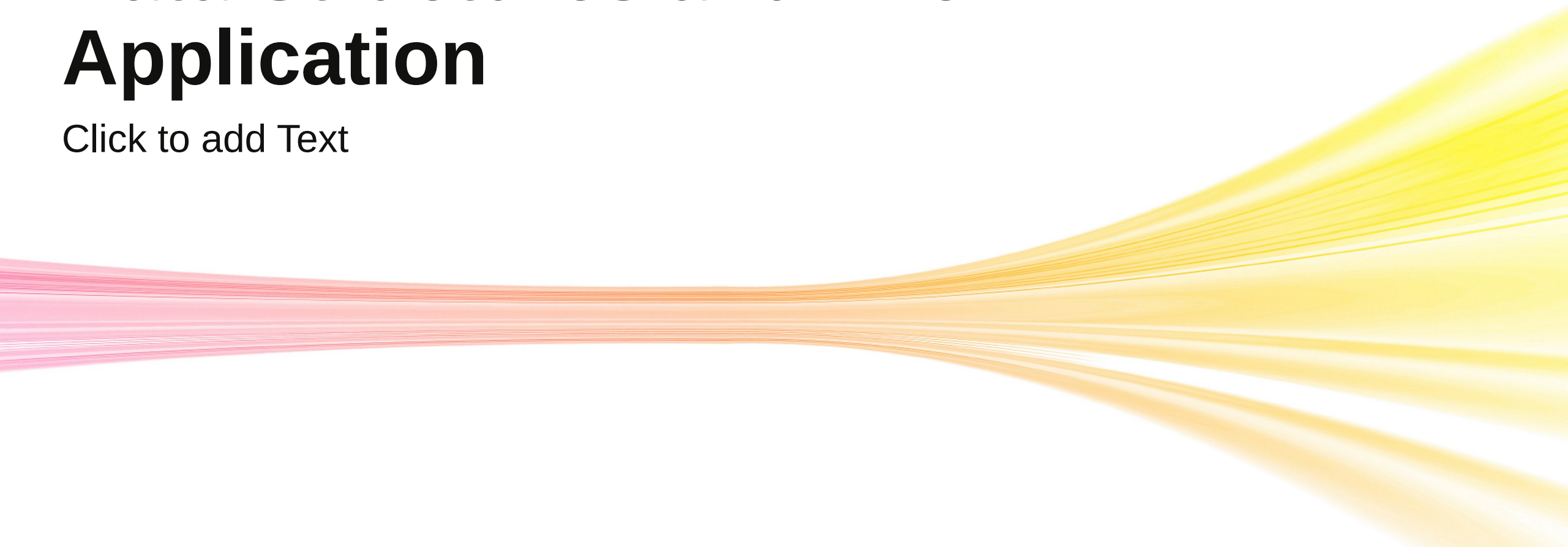
- Analyse differences between arrays, List, and Dictionary

Evaluate

- Evaluate structure suitability based on problem context

Data Structures and Their Application

Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that originate from the left side of the slide and extend towards the right. The lines are colored in a gradient of pink, orange, and yellow, creating a sense of motion and energy.

Introduction to Data Structures

Core Concepts

- **List** holds resizable, ordered items:

// List: Ordered and resizable collection

```
List<string> shoppingList = new List<string> { "Apple", "Banana", "Cherry" };
```

- **Arrays** protect fixed-size groupings:

// Array: Fixed-size sequence

```
double[] coordinates = {54.97, -1.61};
```

- **Dictionaries** power lightning-fast lookups:

// Dictionary: Key-value map pairs

```
Dictionary<string, object> userProfile = new Dictionary<string, object> { ["id"] = 101, ["role"] = "Admin" };
```

Structural Differences

Feature	List (< >)	Arrays ([])	Dictionary ({ })
Ordering	Ordered sequence	Ordered sequence (fixed positions)	No guaranteed order
Mutability	Mutable (Resizable)	Fixed-size (locked)	Mutable keys/values
Access Mode	Integer index ([0])	Integer index ([0])	Unique Key (["id"])
Duplicates	Allowed entries	Allowed entries	Duplicate keys banned

Syntax Traps

- List uses **angle brackets** for generics

< >

- Arrays use **square** brackets

[]

- Dictionaries use **.Add()** to add key-value pairs

.Add(k, v)

Strengths and Constraints

□ **List: Dynamic Growth**

- **Strength:** Easily append elements
- **Constraint:** Slower lookups in large datasets

□ **Arrays: Fixed-Size Safety**

- **Strength:** Write-protected memory blocks
- **Constraint:** Cannot alter size after initialisation

□ **Dictionary: Key-Value Speed**

- **Strength:** Instant data element lookups
- **Constraint:** Uses significant system memory overhead

Mutability
vs.
Rigid Protection

Real-World Use Cases

☐ **List: Ordered Data Logs**

- Track incoming sensor streaming data
- Manage linear queue processing lists

☐ **Arrays: Grouped Record Constants**

- Store permanent latitude/longitude coordinate data
- Define fixed RGB colour palette values

☐ **Dictionary: High-Speed Lookups**

- Build server status lookup systems
- Manage configuration settings map tables

Mapping Code
to
Business Needs

Right vs. Wrong Collections

Inappropriate Choices:

- **Bad:** Using a List to look up prices by item name (requires checking every item sequentially)
- **Bad:** Using an array for user shopping carts (crashes when adding items)

Appropriate Replacements

- **Good:** Use a Dictionary for rapid keyword lookups
- **Good:** Use a List for fluid cart additions

Complexity Metrics

Structure	Element Access	Element Search	Insertion / Deletion
Array	$O(1)$ (Instant Index)	$O(n)$ (Linear Scan)	$O(n)$ (Requires Resize / Copy)
List	$O(1)$ (Instant Index)	$O(n)$ (Linear Scan)	$O(1)$ average at end / $O(n)$ if shifted
Dictionary	$O(1)$ via Key	N/A (Handled via Key)	$O(1)$ (Constant Hashing Time)



FIREBRAND

LYCEUM EDUCATION GROUP

Hands-On Exploration

Practical Exercises: Manipulating Collections

Core Tasks:

- **Modify collections:** Add, remove, and alter elements inside a List
- **Update Dictionaries:** Insert new key-value pairs and change records
- **Access sequences:** Extract elements using numeric indexes and unique keys
- **Loop collections:** Traverse all data structures using iterative structures

Expected Outcome:

- **Working scripts:** Programs execute smoothly without throwing exceptions
- **Correct types:** Code utilises the correct data structure for each task

Task 1: List Operations

```
// Create an inventory log List  
List<string> inventory = new List<string> { "Laptop",  
"Monitor", "Keyboard" };  
  
// TODO: Add "Mouse" to the end of the inventory list  
// TODO: Remove "Monitor" from the collection  
// TODO: Update the first item in the list to "Premium  
Laptop"  
  
Console.WriteLine("Updated Inventory: " + inventory);
```

Mechanics to Observe:

- **Zero-indexing:** Accessing the first item requires index 0
- **Dynamic resizing:** List size grows automatically after modifications

Task 2: Dictionary Lookups

```
// Define a server profile database entry
Dictionary<string, string> server = new
Dictionary<string, string> { {"host", "192.168.1.1"},
{"status", "Online"} };

// TODO: Add a new key "port" with the value "8080"
// TODO: Update the "status" key to "Maintenance"
// TODO: Access and print the "host" value using its key

Console.WriteLine("Updated Server Data: " + server);
```

Mechanics to Observe:

- **Key collisions:** Assigning a value to an existing key overrides the old data
- **Banned indexing:** Attempting `server[0]` will fail unless 0 is an explicit key

Task 3: Iteration Experiments

```
// Array of server location coordinates (2D array)
double[,] locations = {{54.97, -1.61}, {51.50, -0.12},
{53.48, -2.24}};

// TODO: Use a for loop to iterate over the locations
array

// TODO: Inside the loop, extract and print the
individual coordinates
```

The Fixed-Size Array Challenge:

- **Experiment:** Try to access a fourth coordinate that doesn't exist (e.g., `locations[3, 0]`)
- **Observation:** Note the exact `IndexOutOfRangeException` crash that C# throws to protect the fixed-size structure

Stacks vs. Queues

The Stack (LIFO):

- **Principle:** Last-In, First-Out (LIFO)
- **Core Mechanics:** Elements are added (Push) and removed (Pop) from the exact same end (the top)
- **C# Implementation:**

```
Stack<string> history = new Stack<string>();
```
- **Real-World Use:** Browser "Back" button navigation or text editor "Undo" stacks

The Queue (FIFO):

- **Principle:** First-In, First-Out (FIFO).
- **Core Mechanics:** Elements enter at the back (Enqueue) and exit from the front (Dequeue).
- **C# Implementation:**

```
Queue<int> printBuffer = new Queue<int>();
```
- **Real-World Use:** Operating system printer queues, background task processing, or server request buffering

Linked Lists

The Concept:

- Unlike primitive arrays, a **Linked List** is not stored sequentially in sequential memory blocks
- It is an abstract chain composed of isolated Nodes

Anatomy of a Node:

- **Value:** The actual data being stored (e.g., an integer or string)
- **Pointer:** A reference property linking to the Next node in the sequence

Engineering Trade-offs:

- **The Benefit:** Modifying data elements takes constant time $O(1)$ because you only have to update pointers, avoiding expensive memory copies
- **The Risk:** Checking items requires a sequential linear search scan $O(n)$ from the Head node downward because index-based random access is not available
- **C# Syntax:** `LinkedList<double> route = new LinkedList<double>();`

Trees, Graphs, & Hash Tables

Trees (Non-Linear Hierarchy):

- Organises data into explicit parent-child nodes branching downward from a single Root node
- **Binary Search Trees (BST):** Left children are always smaller than the parent; right children are larger. This enables searching in logarithmic time ($O(\log n)$)

Graphs (Interconnected Networks):

- A set of data nodes called Vertices connected dynamically by relational links called Edges
- *Real-World Use:* Network routing tables, social media friend connections, and mapping GPS coordinates

Hash Tables (Constant-Time Mapping):

- The underlying concept powering C# Dictionaries
- Passes a human-readable key into a mathematical Hashing Function to calculate an explicit array memory index, matching records in instant constant time ($O(1)$)

CRUD Operations & Summary

Retrieval, Updates, and Collection Iteration

- **Retrieve:** Safely check key presence before extracting quantities
- **Update:** Target nested value coordinates to mutate state
- **Iterate:** Use foreach to unpack parent and child fields together

Core Iteration Pattern:

// Unpacking keys and nested containers cleanly

```
foreach (var entry in inventory) {  
    Console.WriteLine(entry.Key + ": " + entry.Value["qty"]); }  
}
```

Lab Activity: Stock-Tracking Prototype

The Mission:

Building a Scalable Nested Inventory Structure

- **Goal:** Build a functional inventory dashboard engine
- **Core Collection:** Implement a nested Dictionary pattern
- **Mapping Rule:** Map item string keys to inner property Dictionaries

Structural Strategy

// Item Name Key -> Maps to inner attribute
Dictionaries

```
Dictionary<string, Dictionary<string, object>> inventory  
= new Dictionary<string, Dictionary<string, object>>();  
inventory.Add("Laptop", new Dictionary<string, object>  
{  
    ["price"] = 899.99,  
    ["qty"] = 15  
});
```

Inventory Management Prototype



ACTIVITY



Objective: Build a small stock-tracking prototype

Tasks / Requirements:

- Store item data using a Dictionary of Dictionaries (item name -> {price, qty})
- Add functionality to retrieve and update a quantity
- Iterate through the inventory to print a summary

Key Point: Justify why a Dictionary was chosen over a List for this scenario

Extension Activity: Add additional fields such as category or supplier



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation and Comparison Discussion

Analysing Data Structure Trade-Offs

The Three Evaluation Pillars

- **Flexibility vs. Structure:** Balancing the ability to mutate data easily against rigid structure enforcement
- **Read vs. Write Efficiency:** Analysing how choice of collection alters processor speed when looking up or saving data
- **Large-Scale Maintainability:** Evaluating how code adapts when data sets grow from three entries to three million

Expected Outcome

- **Explain Trade-Offs:** Articulate precisely why a chosen structure excels in one area while introducing constraints in another

Flexibility vs. Rigid Structure

Mutable Flexibility (List & Dictionary):

- **The Good:** Dynamic scaling allows items to be added, removed, or updated on the fly
- **The Risk:** Free-form editing leaves records open to accidental overwrite bugs or data type inconsistencies

Structural Protection:

- **The Good:** Write-protected memory blocks guarantee data remains unchanged throughout program lifecycles
- **The Risk:** Absolute immutability requires copying and recreating objects if values ever need to change (**Arrays**)

Read vs. Write Efficiency

Sequential Scans (List):

- **Write:** Fast - items simply append straight to the end of array memory
- **Read:** Slow - finding a specific item forces a Linear Time check through every single slot

Direct Hashing Maps (Dictionary):

- **Write:** Slight memory overhead needed to compute unique internal hash keys
- **Read:** Instant - fetching by direct key identifier scales at lightning-fast Constant Time

Maintainability in Larger Systems

**Schema
Rigidity**

**Team
Readability
Impact**

Stretch and Challenge

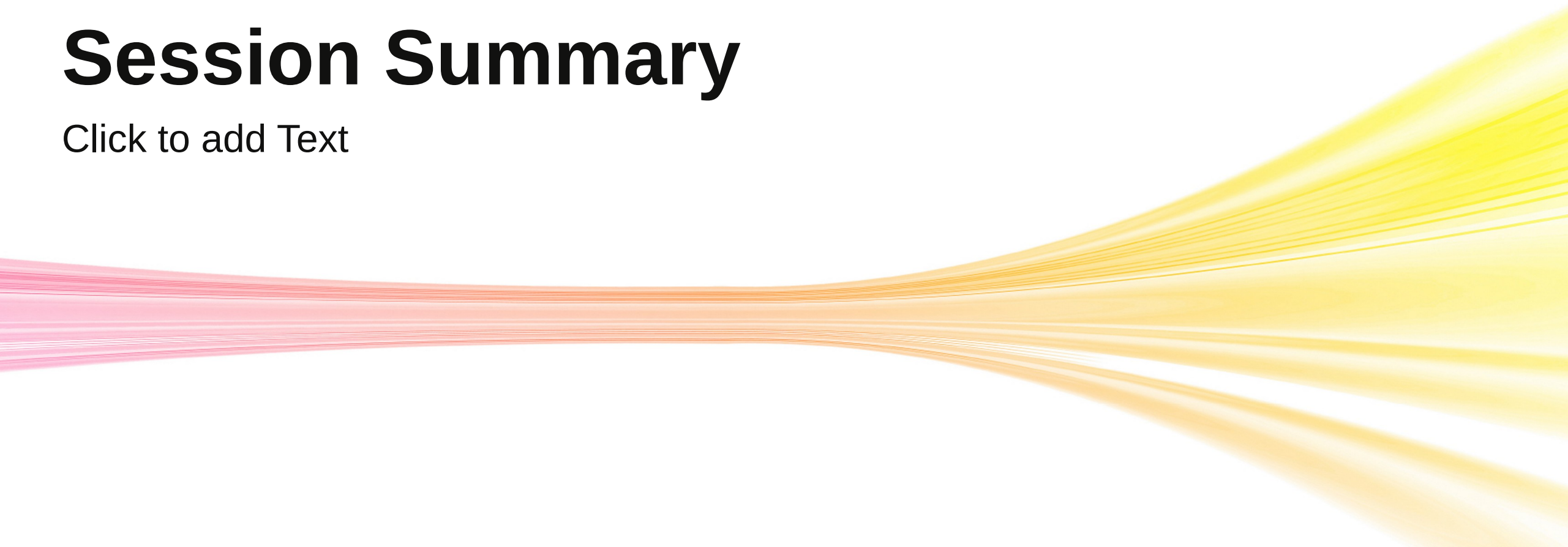
The Mission:

Upgrade your Inventory Management Prototype into production-grade code

- **Task 1:** Explore abstract structures such as stacks and queues
- **Task 2:** Map these concepts to C# implementations using List or LinkedList
- **Task 3:** Explain how these patterns are used in real-world applications

Session Summary

Click to add Text



Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Data Structures and Their Application

Lesson Objectives

Apply

- Apply appropriate data structures to store and organise data

Analyse

- Analyse differences between arrays, List, and Dictionary

Evaluate

- Evaluate structure suitability based on problem context

